

NextIn

A Virtual Queue App

YOUR PLACE IN LINE, RIGHT ON YOUR PHONE

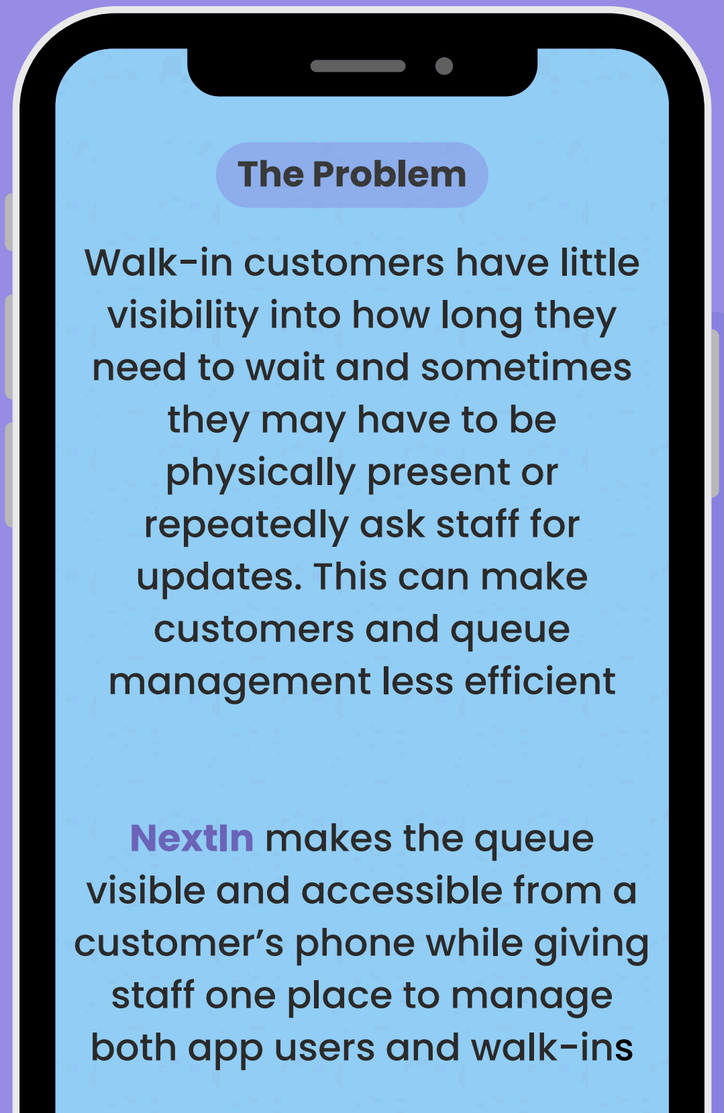
Suchir Jolapara - U08469805
Namratha Reddy - U65966062

Project Concept & Problem

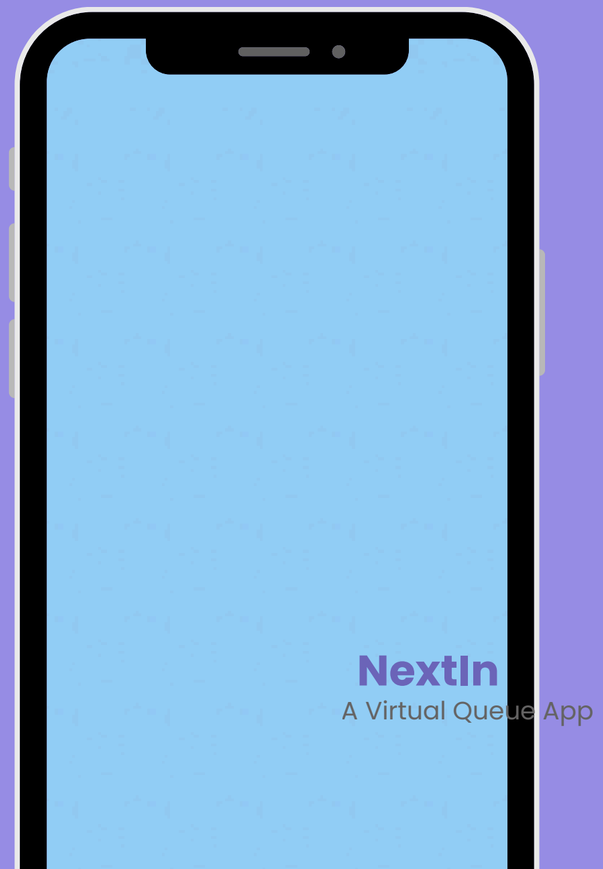
NextIn is a virtual queue app that allows customers to join a business's waiting list, view their live position, estimated wait time, and leave the physical waiting area without losing their place. Businesses use a separate staff interface to manage the same queue in real time.

Why Mobile?

Customers can track their queue status wherever they go and receive timely updates as their turns approach



Users & Value



1

Customers

Walk-in customers at various businesses such as salons and other service based businesses

2

Business Staff

Employees responsible for managing the business's waiting line

3

Value

Customers gain freedom and visibility while waiting. Businesses gain a shared, organized queue and spend less time answering status questions

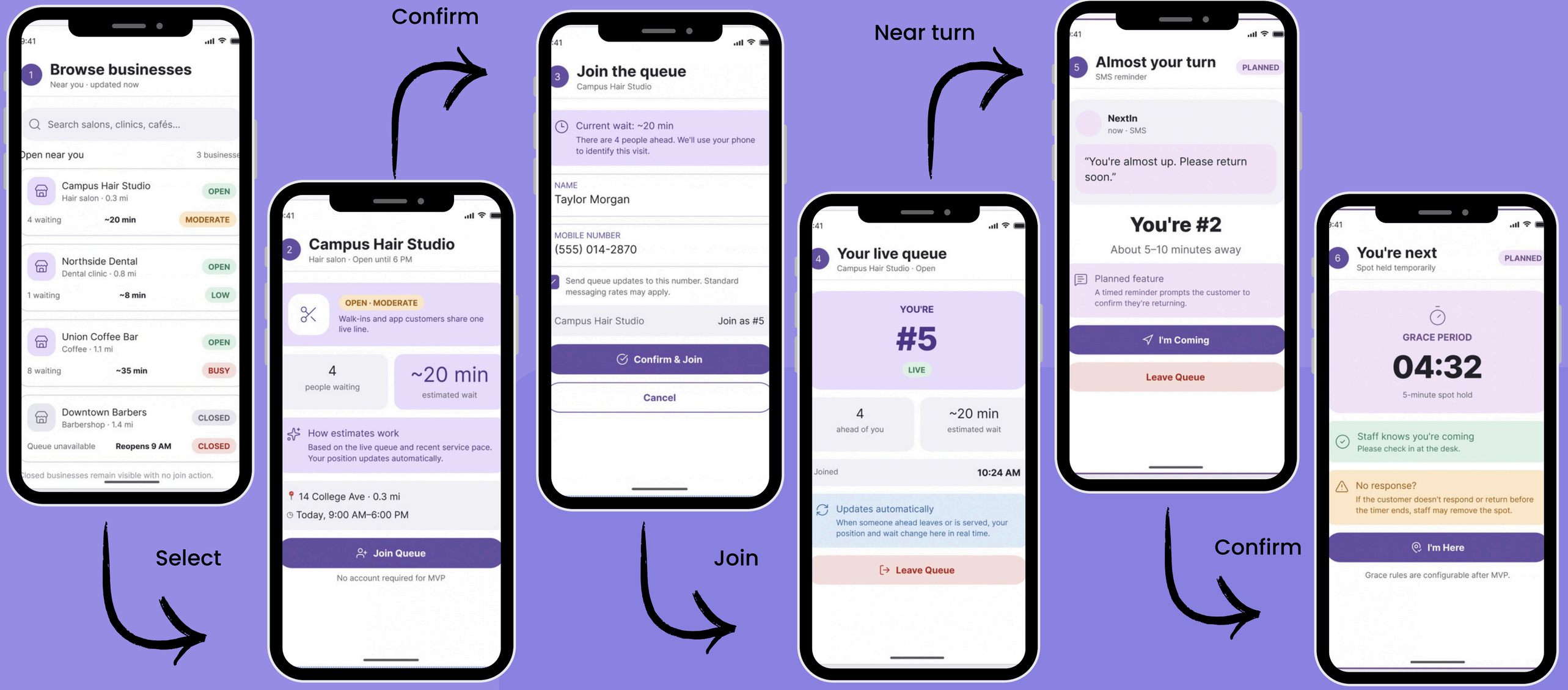
4

Current Assumptions

Customers have internet access, while staff have authority to manage the queue. Initial businesses primarily use first-come, first-served walk-in queues

User Experience

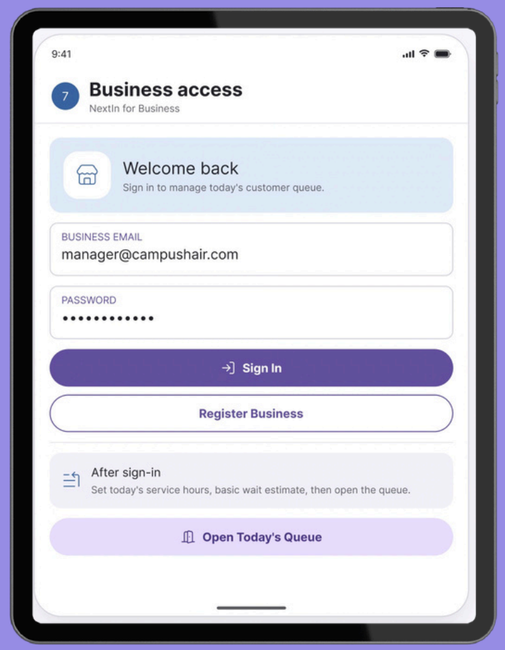
MVP + Customer Journey



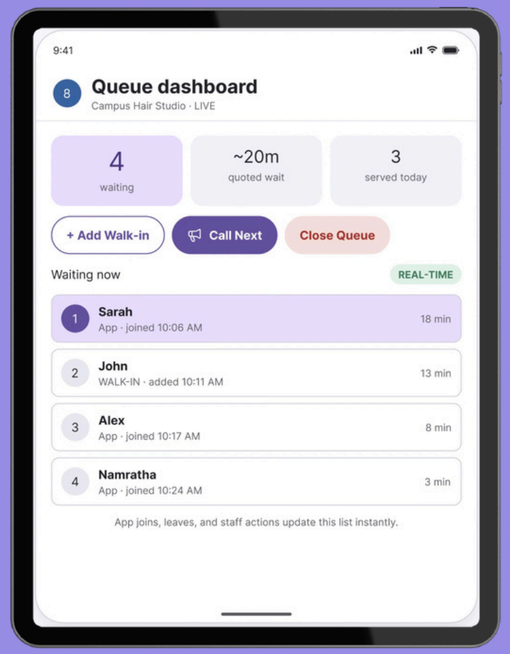
User Experience

MVP + Business Journey

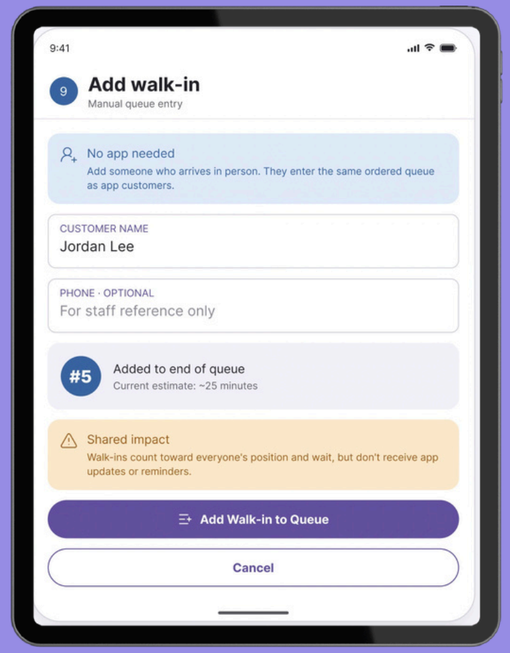
*Initial wait estimate = people ahead × business's average service time



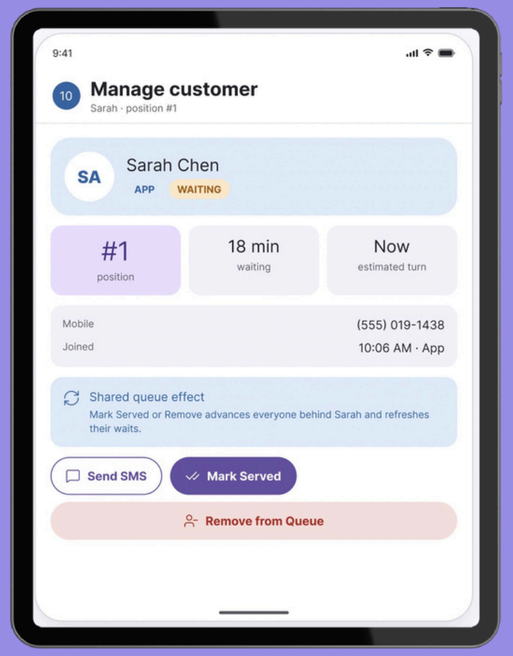
open



add



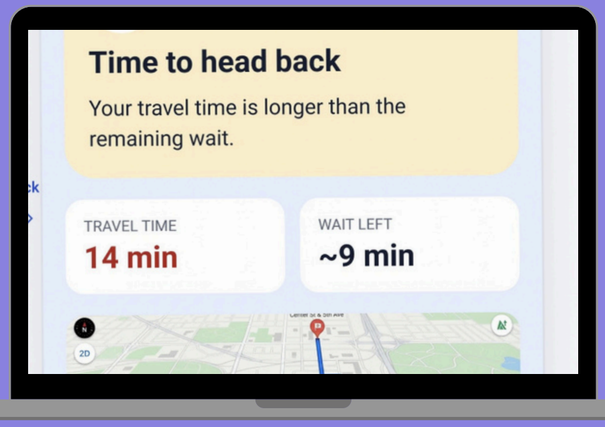
select

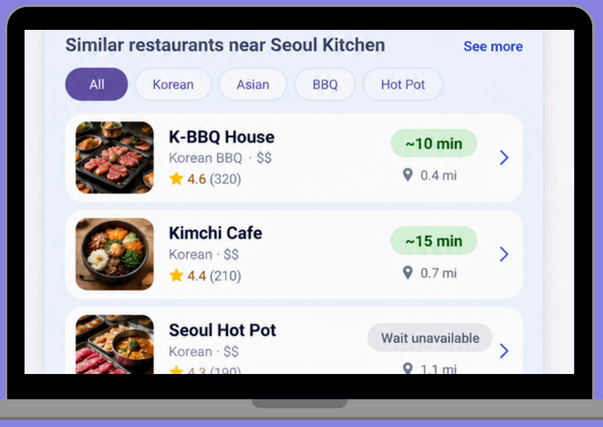


Future Scope

- Queue rules (restaurant party size, different services)
- Other recommendations (similar stores)
- Location-aware reminders

(These features will only be attempted after the core real-time queue experience is complete)





Technical Approach

Kotlin + Jetpack Compose: Customer and staff interfaces

Navigation Compose: Screen navigation and role-based flows

ViewModel + StateFlow: Lifecycle-aware UI state and live queue updates

Data & Services

- Firebase Authentication: account/role authentication
- Cloud Firestore: queue data + real-time synchronization
- Repository layer: separates UI from Firebase

Still to Research

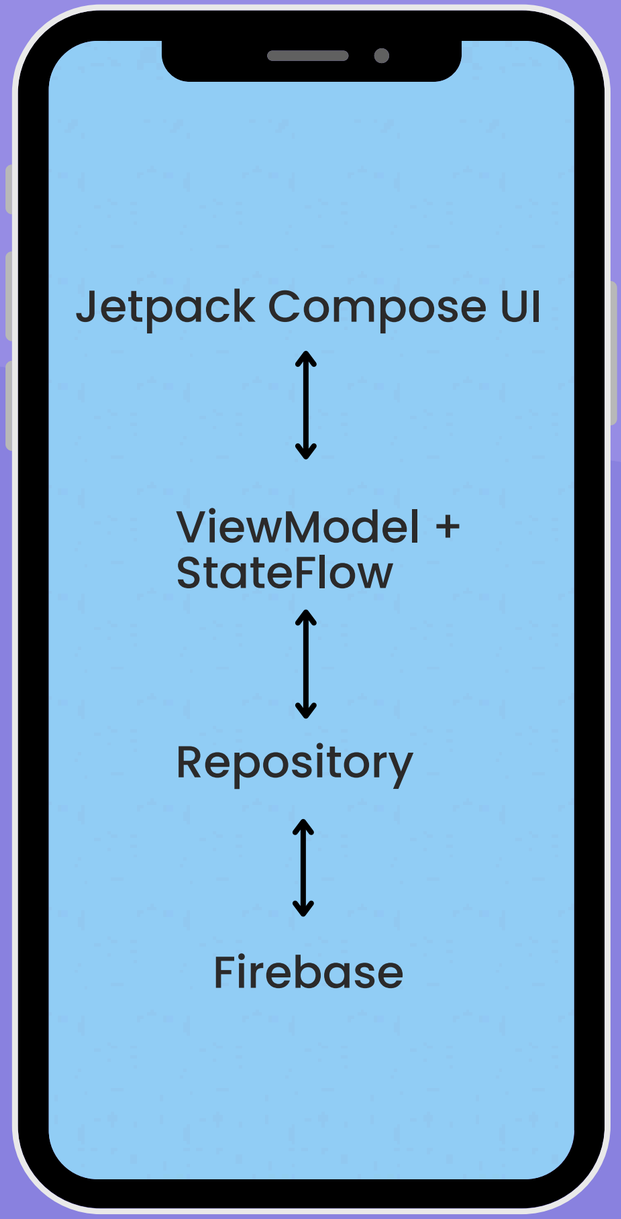
- Customer authentication vs. guest entry
- Concurrent queue updates & Firestore security rules
- Notification implementation

Local persistence

- Local caching/persistence will be evaluated if needed

For device capabilities

- No location, sensors, or wearable integration is required



Team & Development Plan

Namratha

- Customer-side UI & navigation
- Queue status/state
- UI testing

Suchir

- Staff-side UI
- Firebase/data model
- Authentication

Shared

- Integration
- Architecture decisions
- Code review & final testing

(split may adjust based on project direction)

Collaboration:

GitHub Issues → Feature Branch → Pull Request → Review → Main

GitHub link: https://github.com/namz0516/CS501-NextIn.git

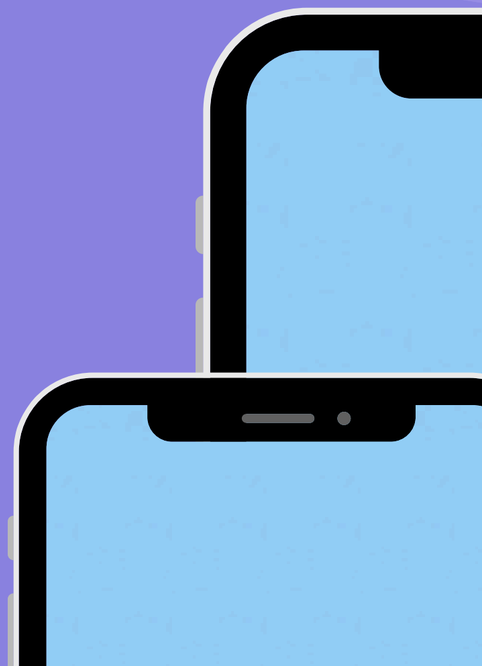
Communication: WhatsApp, Google Meet, in-person discussions (2x a week)

Scrum Master: Responsibilities to be rotated bi-weekly

Development approach: Agile with weekly iterations

Initial Backlog:

- Week 1: Set up Compose + navigation
- Week 2: Build customer queue
- Week 3: Build staff dashboard
- Week 4: Connect Firebase + authentication
- Week 5: Implement join/leave/serve
- Week 6: Real-time queue updates
- Week 7: Notifications + optional features
- Week 8: Test application, bug fixes, final polish



Risks & Questions

Technical & Security:
Simultaneous joins or staff actions could create inconsistent queue positions

Plan: Firestore transactions, authentication, and security rules to control queue updates

Scope, Privacy & Usability:
Notifications and additional features may increase development time, storing customer phone numbers introduces privacy issues, and estimated wait times may not be accurate

Plan: Complete the core queue first, keep location and other features outside the MVP

Questions:

1.) How is our current MVP?

2.) Should we add the “recommend” other businesses feature to our MVP?

3.) For our MVP, can we continue with our FCFS queue logic, or should we have a custom system logic that businesses can define?

References & AI Disclosure

Sources



1. Android Developers. “Jetpack Compose.” Android Developers.
2. Android Developers. “Navigation with Compose.” Android Developers.
3. Google Firebase. “Cloud Firestore.” Firebase Documentation.
4. Google Firebase. “Firebase Authentication.” Firebase Documentation.
5. Google Firebase. “Firestore Security Rules.” Firebase Documentation.
6. Google Maps Platform. “Places API.” Google for Developers.

AI



- ChatGPT was used to help with brainstorming ideas, app name suggestions, discuss limitations to define the scope and assess technical decisions
- Figma was used to assist with the UI design

(Anything else to be used by AI will be disclosed)

NextIn
A Virtual Queue App



Thank You!

Coming soon....